

Backup — How simTIM relates to other simulators

Attack graphs	static reachability; no time, no defender agency
RL cyber gyms	turn-based; an action resolves within one step
Adversary emulation	executes on real hosts; not what-if
Temporal security games	continuous time, but no topology or assets
Model-based evaluation	time <i>and</i> structure — closest relatives

They model **time without a network**, or a **network without time**, or **reality without what-if**. simTIM combines them — and uniquely lets a defence invalidate an attack that is *already running*.

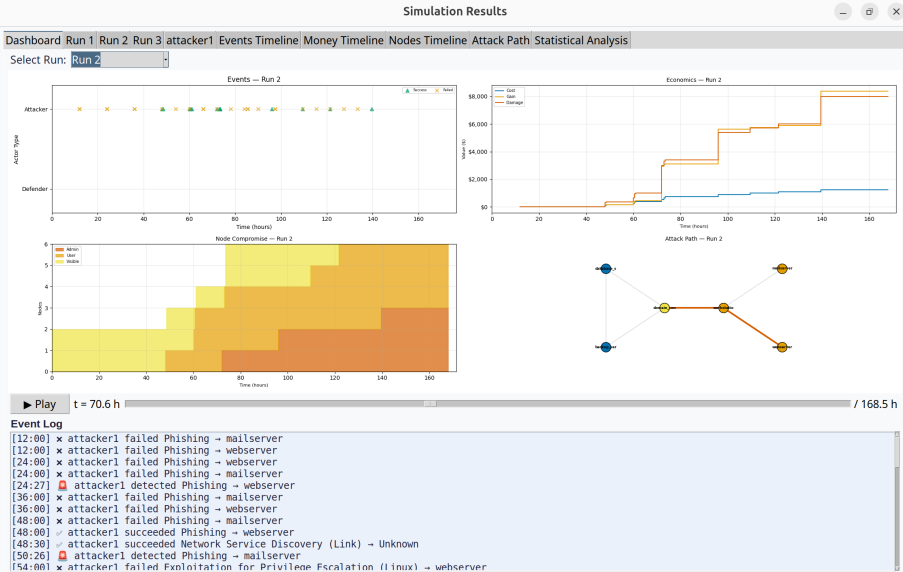
Backup — Why TIM, and not an existing model

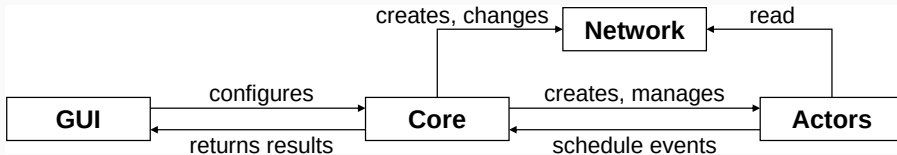
Table 1. Overview of the extent to which existing approaches satisfy the requirements of Sec. 2. ✓: requirement is satisfied; (✓): partially satisfied; –: not at all satisfied

Work	R1	R2	R3	R4	R5	R6	R7	R8	R9
Clempner [3]	–	–	✓	–	–	–	✓	(✓)	✓
Farhang & Grossklags [4]	–	–	(✓)	–	–	✓	(✓)	✓	–
Hoffmann [5]	–	–	–	(✓)	(✓)	–	–	(✓)	–
Merlevede et al. [10]	–	–	(✓)	–	–	–	(✓)	(✓)	–
Outkin et al. [11]	–	–	(✓)	(✓)	–	✓	✓	(✓)	–
van Dijk et al. [12]	–	–	(✓)	–	–	–	(✓)	(✓)	–
Zhang et al. [15]	✓	–	(✓)	(✓)	–	–	(✓)	(✓)	–
Żychowski & Mańdziuk [17]	✓	–	(✓)	(✓)	✓	–	(✓)	(✓)	–
TIM (this work)	✓	✓	✓	✓	✓	✓	✓	✓	✓

Mann, “Time is money”, 2025 — Table 1. R1–R9 are the requirements of that paper; R8 is action cost and duration.

Backup — Demo fallback





Two-step process: a Bernoulli trial with probability ϱ decides *whether*; a CDF $F_a(t)$ on the normalised progress $t \in [0, 1]$ decides *when*.

- **Uniform** $F_a(t) = t$
- **Early-weighted** $F_a(t) = 1 - (1 - t)^n$
- **Late-weighted** $F_a(t) = t^n$

Timing matters: a late detection may arrive after the damage is done. New engine = one subclass with three methods.

Attacker

- escalation — climb access levels, then scan
- greedy — maximise $p \cdot \text{gain} - \text{cost}$
- random — baseline

Defender

- reactive — evict and isolate
- proactive — harden
- monitoring — detect and deceive
- balanced — all tactics

Deliberately simple. The interface, not the heuristics, is the contribution.

- simTIM is a **what-if** tool, not a predictor. It compares configurations under identical assumptions.
- Parameter sources: CVE/EPSS, MITRE ATT&CK, internal incident data, expert estimate.
- Uncertain parameters are *swept*, not guessed — that is what the scenario comparison is for.
- Empirical validation against real incident data: future work.

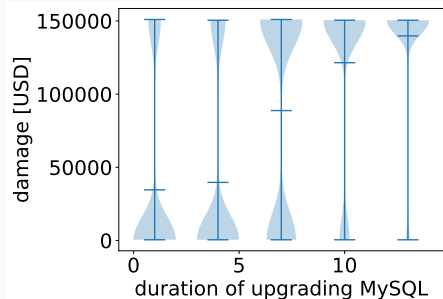
- Discrete-event simulation: cost scales with the number of *events*, not with simulated time.
- Event queue is a binary min-heap (`heapq`).
- An action index makes precondition re-checks cheap when a node changes.
- Monte Carlo runs are independent and can be threaded.
- Not yet benchmarked on very large enterprise graphs.

From the TIM running example:

- Attacker wins: +\$98 600 / defender −\$150 000
- Defender wins: attacker −\$600 / defender −\$500

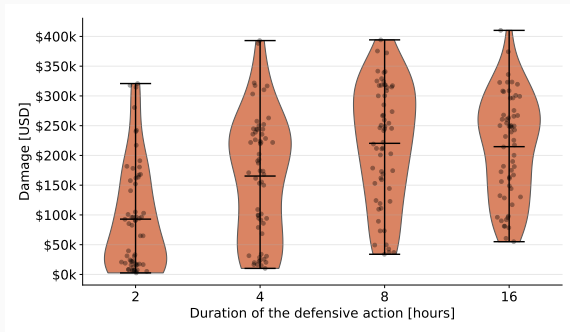
Default durations: 4 h per attack action, 8 h for the defensive action. Detection probability on n_1 : $\varrho = 0.8$ (Sophos present).

Backup — Reproducing the TIM paper result



TIM, 2025

custom code, 2 nodes

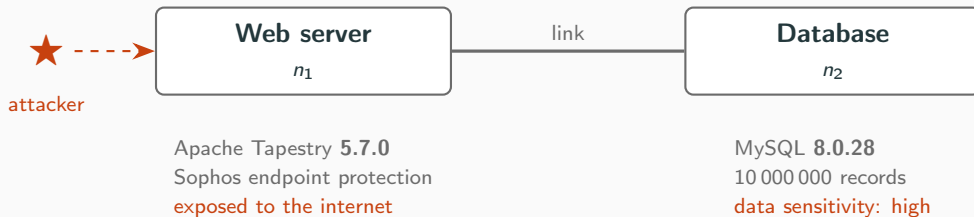


simTIM

a dropdown, 6 nodes, 125 runs

Same finding. Any model you like.

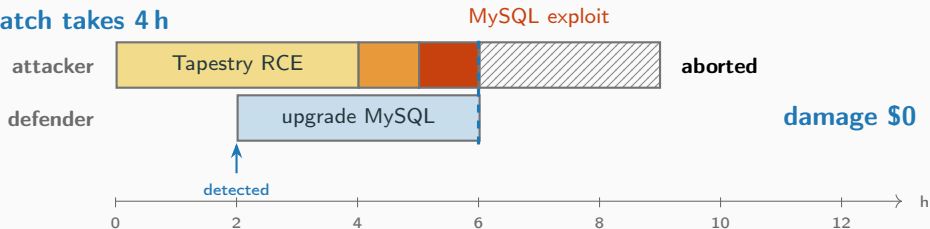
Backup — The TIM running example



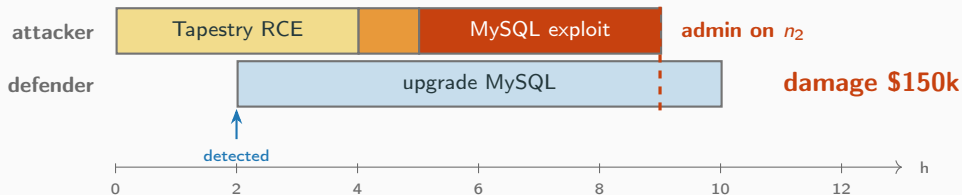
One known RCE. One database worth **\$150 000**. One question:
where does my next euro do the most good?

Backup — The race, in detail

Patch takes 4 h

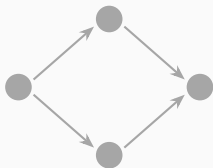


Patch takes 8 h



Nothing changed but **one duration**.

Backup — Why not attack graphs or game theory?



Attack graphs
reachability,
no clock

u_{ij}	u_{ij}
u_{ij}	u_{ij}

Game theory
players move
simultaneously



Severity scores
one static
number

Time is the missing dimension.

TIM closes that gap.

Mann, "Time is money", 2025

TIM

a way to describe

nodes, links, properties

who has which access

actions that take **time**

preconditions that must hold

the whole way through

simTIM

a way to play it out

schedules every action in time

rolls the dice, times the alarms

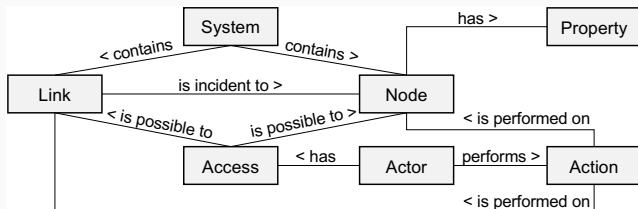
counts the money on both sides

and does it **a thousand times**,

because once tells you nothing

A set of rules does not tell you **who wins**.

Backup — TIM mapped onto the example



the web server → a **Node**

“Tapestry 5.7.0” → a **Property**

the attacker’s foothold → **Access**

the exploit → an **Action**

and every one of them
changes over time

Backup — What an action contains

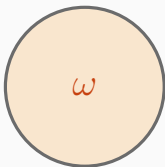
a_{Tapestry} MITRE ATT&CK T1190 · CVE-2021-27850

precondition φ_a	framework = Tapestry $\wedge$ version $\in \{5.4.5, \dots, 5.7.0\}$
postcondition ψ_a	$\omega_x(n) = \text{admin}$
cost c_a	\$300
duration d_a	1 day
success p_a	0.2
detection ϱ	0.8

The precondition must hold **for the whole duration**.

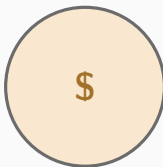
That single rule is what made the previous slide possible.

Backup — The three quantities tracked



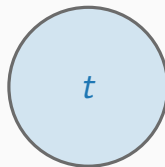
Security

none $\rightarrow$ visible
 $\rightarrow$ user $\rightarrow$ admin



Economics

cost, gain, damage
one-off & over time



Time

duration, capacity,
detection delay

Every run is stochastic $\Rightarrow$ Monte Carlo, always.

Backup — What makes simTIM different

A defence can stop an attack that is already running

everywhere else an action, once started, simply resolves

Continuous time

not turns —
attack and defence
durations differ freely

Detection timing

when it is noticed,
not just whether

Money, both sides

gain $\neq$ damage;
one-off and
time-proportional